

手続き型プログラミング発展 実習 期末レポート

学籍番号：24B_30495

選択した課題：課題 3

氏名：黄海ガ

1 ハフマン符号に関する説明

1.1 歴史

ハフマン符号 (Huffman Coding) は、1952 年に David A. Huffman によって考案された可逆データ圧縮アルゴリズムで、文字の出現頻度に基づいて可変長の符号を割り当て、データサイズを削減するために用いられた符号である

1.2 基本原理

頻出文字に短い符号を割り当て、稀な文字に長い符号を割り当てることで通常の全文字同じ長さでは実現できない保存手法。ただし、解凍時に復元可能である必要があり、たとえば $A=0, B=1, C=10$ のような割当は不可 (10 は BA もしくは C であるため)。復元可能な最適な割り当てを作るためハフマン木 (Huffman Tree) が考察された

1.3 具体例

入力テキスト: "ABRACADABRA"

文字頻度:

A: 5 回 (45.5%)

B: 2 回 (18.2%)

R: 2 回 (18.2%)

C: 1 回 (9.1%)

D: 1 回 (9.1%)

ハフマン符号:

A: 0 (1 ビット)

B: 10 (2 ビット)

R: 11 (2 ビット)

C: 110 (3 ビット)

D: 111 (3 ビット)

元のデータ: 11 文字 $\times$ 8 ビット = 88 ビット

圧縮後: $5 \times 1 + 2 \times 2 + 2 \times 2 + 1 \times 3 + 1 \times 3 = 20$ ビット

圧縮率: 77.3

符号の割り当て方法:

頻度順に並ぶと、D1, C1, R2, B2, A5 になり。その頻度の小さい順に新しい root を作る。

そうすると D1, C1, R2, B2, A5 – root(CD)2, R2, B2, A5 – B2, root(CDR)4, A5 – A5, root(CDRB)6 – root(CDRBA)11 になる

生成された木から逆に文字を探し、A5, root(CDRB)6 – root(CDRBA)11 の左部分を 0 右部分を 1 とすると、A は 0, root(CDRB) は 1 になる。B2, root(CDR)4 – root(CDRB)6 であるため B2 は root(CDRB) と 0、つまり 10、root(CDR) は 11。

1.4 可逆の原因

Huffman Tree の構造上、全ての文字は葉の部分にあり、root になっていないため他の文字の先頭になっていない

2 圧縮アルゴリズムの説明

2.1 全体像

inputFile を開いて各文字の出現回数を記録し、その後出現回数が 0 の文字を取り除き、昇順で queue のようなデータ構造で並ばせる。その queue の最も前、つまり出現回数が最も小さい二つを取り出し、Huffman Tree を作らせる。新たに生成された root をまた queue に入れる。上記の動作を queue に一つのデータしか残っていないまで繰り返すと、Huffman Tree が完全に生成される。生成された木の左と右を 0 と 1 に対応させ、各文字に code を付ける。更に、inputFile を rewind(inputFile) で先頭に戻し、もう一度読み直す。今度は各文字をその文字に付けられた code に置き換え、outputFile に書き込む。また、復元時のことを考えると、code と各文字の対応関係、もしくは各文字の出現頻度を記録する必要がある。今回のプログラムでは後者を採用した。理由は後者の方が outputFile が小さく、圧縮率を高められると思ったためだ。しかし、後者の場合、出現頻度から Huffman Tree を再構築する必要があり、やや解凍に時間かかるデメリットも存在する。

2.2 個別の説明

2.2.1 FileHeader の説明

FileHeader は各ファイルの名前、元のサイズと圧縮後のサイズ、圧縮された時間のような基本的なデータを記録している。それらは全てアーカイブに保存する。保存時は File1 の FileHeader、File1 の内容、File2 の FileHeader、File2 の内容の順に保存する。理由は FileHeader もしくは File の内容をメモリで保存したらメモリの浪費に繋がるためファイル一つずつ書き込む。

2.2.2 Struct node の説明

今回のプログラムでは、node は Huffman Tree の一つの node に対応する。char ch はその node の対応する文字であり、root、つまり他の二つの node により作られた場合、対応する文字が無い場合 '\0' に設定する。next は node に quene の構造を持たせるために設定したもので、最初は node とは別に struct quene を作り、それに node* data と quene* next を持たせようとしたが、quene->data->freq の参照と比べるのが面倒なのと、実際 node に全部統一した方が効率もいいと思い、全部 node に書き込んだ。また、next と prev 両方を node に持たせるのも可能だが、今回のプログラムでは prev が無くても大して差は無さそうなので next だけ持たせた。

2.2.3 frequencyList の説明

frequencyList の index で各文字に対応させ、その value はその文字が出現した回数に対応する。例えば A=65 であるため、A が出現するごとに frequencyList[65]++ を実行し、その出現を記録する

2.2.4 insertNode の説明

newNode と今の*head の freq を比べ、newNode が*head より前に並ぶ場合、head も更新する。それ以外の場合は newNode が今のポインター、つまり temp の next の node の前か後ろのどちらに並ばせるべきかを判断する。ただし、temp の next の node が存在しない場合は、temp の後ろにそのまま並ばせる

2.2.5 compressFileToArchive の説明

このプログラムでは rb,wb のような二進法でファイルを読み取るもしくは書き込む。この関数は特殊な場合、つまりファイルが空もしくは全部同じ文字である場合を考慮した。上記以外の場合は、fileSize でファイルのサイズを保存し、それに応じて fileData がメモリを確保し、一気にファイルを全部読み込む。frequencyList で fileData の中身、つまり各文字の出現回数を記録する。codes がこのプログラムにおける唯一のグローバル変数であるため、先に全部初期化する。また、8bit = byte であるため compressedSize を計算する際、8 で割った。もし小数が存在する場合、1 を足す。上記が終わった後、FileHeader にデータを書きこむ。最後に、FileHeader のデータ、復元用の frequencyList および肝心の圧縮後のデータを archive に書き込む。これで compress の作業が終わり、必要なデータを print out してメモリをも解放し、return 1 で終わる

3 解凍アルゴリズムの説明

最初に outputDir が与えられたかどうかを判断する。与えられた場合、その outputDir が実在するかどうかを判断する。全部満たす場合のみ Destination を print out する。その後、archive ファイルを開き、そこに保存された圧縮後のファイルの一つずつ decompressFileFromArchive.archive, outputDir, 1) を用いて解凍する。decompressFileFromArchive 関数では、archive ファイル先頭にある frequencyList を読み取り、圧縮時と同様に Huffman Tree を再構築する。手法が同じなため構築された Huffman Tree も同じである。左を 0、右を 1 とし、圧縮されたファイルから 0 もしくは 1 を一つずつ読み込み、root から辿ると最終的に到達した node が対応する char を inputFile に書き込む。それを繰り返すことで最終的には解凍されたファイルのサイズが記録された FileHeader の originalSize と等しくなり、つまり圧縮されたファイルの一つが完全に解凍される。archive file の先頭にファイルの総数も書かれてあるため、それに応じて循環の回数を決めれば最終的には全部のファイルが解凍される。

4 圧縮ファイル形式の説明

圧縮ファイルの先頭には圧縮されたファイルの総数が書かれている。その後一つ目の File の FileHeader と圧縮後のデータが書かれている。それに続いて二つ目の File の FileHeader と圧縮後のデータと、FileHeader と圧縮後のデータの二部分が繰り返す。FileHeader はこのプログラムで作った struct で、その内容は file の名前という文字列と、元のサイズと圧縮後のサイズ、変更された時間 `time_t` `modifiedTime` が保存されている。圧縮後のデータは文字を一つずつ Huffman Tree に基づいて生成された code で変換したあとのデータである。

5 プログラムについて

5.1 プログラムの説明

プログラムには create,extract,list の機能が実装されており、create でファイルをアーカイブファイルに圧縮でき、extract でアーカイブファイルからファイルを解凍することができる。list でアーカイブファイルから圧縮されたファイルや元のファイルの情報を読み取ることができる。

5.2 プログラムのコンパイルの仕方

Terminal で gcc -o compress main.c を打ち込む。ただし、Terminal のアドレスとファイルのアドレスによって cd コマンドでアドレスを変える必要がある。

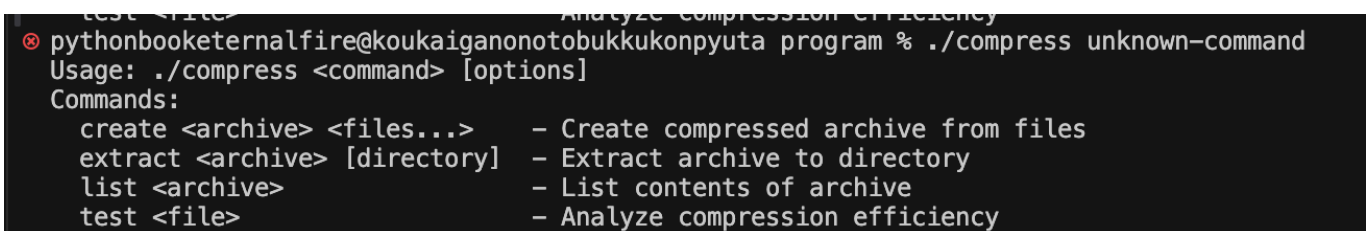
5.3 プログラムの実行方法と想定した出力例

Commands:

create <archive> <files...> - Create compressed archive from files

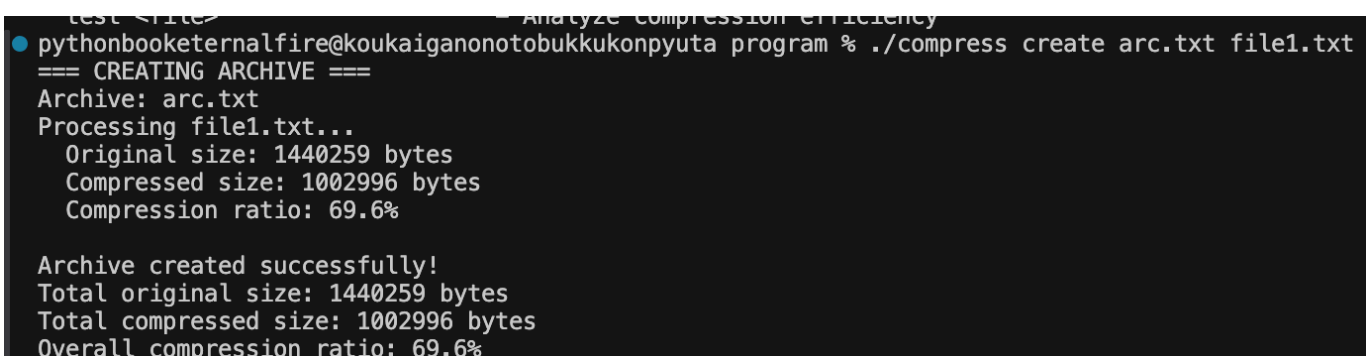
extract <archive> [directory] - Extract archive to directory

list <archive> - List contents of archive



```
pythonbooketernalfire@koukaiganonotobukkukonpyuta program % ./compress unknown-command
Usage: ./compress <command> [options]
Commands:
  create <archive> <files...> - Create compressed archive from files
  extract <archive> [directory] - Extract archive to directory
  list <archive>                - List contents of archive
  test <file>                  - Analyze compression efficiency
```

図 1: Unknow-Command



```
pythonbooketernalfire@koukaiganonotobukkukonpyuta program % ./compress create arc.txt file1.txt
=== CREATING ARCHIVE ===
Archive: arc.txt
Processing file1.txt...
  Original size: 1440259 bytes
  Compressed size: 1002996 bytes
  Compression ratio: 69.6%

Archive created successfully!
Total original size: 1440259 bytes
Total compressed size: 1002996 bytes
Overall compression ratio: 69.6%
```

図 2: create NormalFile


```

pythonbooketernalfire@koukaiganonotobukkukonpyuta program % ./compress create arc.txt fileSingle.txt
=== CREATING ARCHIVE ===
Archive: arc.txt
Processing fileSingle.txt...
  Original size: 14 bytes
  Compressed size: 2 bytes
  Compression ratio: 14.3%

Archive created successfully!
Total original size: 14 bytes
Total compressed size: 2 bytes
Overall compression ratio: 14.3%

```

図 3: create Single Character File

```

eternalfire@koukaiganonotobukkukonpyuta HuffmanArchiver % ./compress create arc.txt src/tests/fileEmpty.txt
=== CREATING ARCHIVE ===
Archive: arc.txt
Processing src/tests/fileEmpty.txt...
Warning: File 'src/tests/fileEmpty.txt' is empty

Archive created successfully!
Total original size: 0 bytes
Total compressed size: 0 bytes

```

図 4: create Empty File

```

pythonbooketernalfire@koukaiganonotobukkukonpyuta program % ./compress create arc.txt file1.txt file2.txt
=== CREATING ARCHIVE ===
Archive: arc.txt
Processing file1.txt...
  Original size: 1440259 bytes
  Compressed size: 1002996 bytes
  Compression ratio: 69.6%
Processing file2.txt...
  Original size: 1440801 bytes
  Compressed size: 1003470 bytes
  Compression ratio: 69.6%

Archive created successfully!
Total original size: 2881060 bytes
Total compressed size: 2006466 bytes
Overall compression ratio: 69.6%

```

図 5: create MultiFiles

```

eternalfire@koukaiganonotobukkukonpyuta HuffmanArchiver % ./compress create arc.txt src/tests/file1.txt src/tests/fileEmpty.txt
=== CREATING ARCHIVE ===
Archive: arc.txt
Processing src/tests/file1.txt...
  Original size: 1428298 bytes
  Compressed size: 990490 bytes
  Compression ratio: 69.3%
Processing src/tests/fileEmpty.txt...
Warning: File 'src/tests/fileEmpty.txt' is empty

Archive created successfully!
Total original size: 1428298 bytes
Total compressed size: 990490 bytes
Overall compression ratio: 69.3%

```

図 6: create Multi-Files with an Empty File

```

pythonbooketernalfire@koukaiganonotobukkukonpyuta program % ./compress extract arc.txt
=== EXTRACTING ARCHIVE ===
Archive: arc.txt
Extracting file1.txt...
  Decompressed: 1002996 → 1440259 bytes
  Saved to: file1.txt

Extraction complete!
1 files extracted successfully

```

図 7: extract Single-File archive

```

pythonbooketernalfire@koukaiganonotobukkukonpyuta program % ./compress extract arc.txt
=== EXTRACTING ARCHIVE ===
Archive: arc.txt
Extracting file1.txt...
  Decompressed: 1002996 → 1440259 bytes
  Saved to: file1.txt
Extracting file2.txt...
  Decompressed: 1003470 → 1440801 bytes
  Saved to: file2.txt

Extraction complete!
2 files extracted successfully

```

図 8: extract Multi-Files archive

```

pythonbooketernalfire@koukaiganonotobukkukonpyuta program % ./compress list arc.txt
=== ARCHIVE CONTENTS ===
Archive: arc.txt
Created: 2025-08-03 08:02:47
Total files: 1

File 1: file1.txt
  Original size: 1440259 bytes
  Compressed size: 1002996 bytes
  Compression: 69.6%
  Modified: 2025-08-02 23:31:25

```

図 9: List

```
● eternalfire@koukaiganonotobukkukonpyuta program % ./compress list arc.txt
=== ARCHIVE CONTENTS ===
Archive: arc.txt
Created: 2025-08-04 09:33:46
Total files: 1

File 1: fileEmpty.txt
  Original size: 0 bytes
  Compressed size: 0 bytes
  Modified: 2025-08-03 07:35:05
```

☒ 10: list Empty File

```
● eternalfire@koukaiganonotobukkukonpyuta program % ./compress list arc.txt
=== ARCHIVE CONTENTS ===
Archive: arc.txt
Created: 2025-08-04 09:39:55
Total files: 2

File 1: file1.txt
  Original size: 1428298 bytes
  Compressed size: 990490 bytes
  Compression: 69.3%
  Modified: 2025-08-04 09:39:44

File 2: fileEmpty.txt
  Original size: 0 bytes
  Compressed size: 0 bytes
  Modified: 2025-08-03 07:35:05
```

☒ 11: list Multi-Files with an Empty File

5.4 工夫点 アピールポイント

5.4.1 unsigned char について

最初は char を使って frequencyList[(char) fileData[i]] ファイルデータの文字が出現した回数を記録しようとしたが、char の範囲が-128 から 127 であるため、範囲が 0 から 255 までの unsigned char を使って frequencyList[(unsigned char) fileData[i]] で回数を記録した

5.4.2 insertNode の中身の一部について

最初は

```
while(temp->next != NULL){
    if(temp->next->freq > newNode->freq){
        newNode->next = temp->next;
        temp->next = newNode;
        return;
    }
    else{
        temp = temp->next;
    }
}
temp->next = newNode;
```

改善後は

```
while(temp->next != NULL && temp->next->freq <= newNode->freq ){
    temp = temp->next;
}
newNode->next = temp->next;
temp->next = newNode;
```

5.4.3 archive ファイルについて

メモリを節約するため、ファイル一つずつ archive ファイルに書き込んだ。そのため、FileHeader の位置がバラバラで、ftell() を使って pointer の位置を計算し工夫した

5.4.4 時間情報の取得について

入出力例では achive ファイルに各ファイルの Modified 時間が書かれている。それを取得するため stat 関数について勉強し、struct stat fileStat を使ってファイルの各種情報を取得した。

また archive ファイルに Created 時間、つまり作成された時間が書かれているため、それを取得するため time.h についても勉強した。

5.4.5 frequencyList について

最初は

```
typedef struct dictionaryPart{
    char ch;
    int freq;
    dictionaryPart* next;
}dictionaryPart;
```

で文字とその出現回数を記録し、文字が増えるごとに新たに dictionaryPart を作る仕組みでプログラムを作ろうとしたが、特定の ch が対応する freq を探すために dictionaryPart* head から一つずつ探す手間があり計算量が大きい且つプログラムが大きくなるためより良い方法を探した。その時思いついたのが、A=65 のように char を 0~255 までの数値に置き換えることができる点だ。それを活用し、その 65 を index とすると、value を探すのがかなり速くなる且つプログラムも簡単化できた